Neural Conjecture Generation for Automated Theorem Proving via Symbol-Anonymous Graph Neural Networks

Anonymous Authors
anonymous@example.com

March 19, 2026

Abstract

We present a neural system for generating useful intermediate lemmas ("cuts") that speed up automated theorem provers. Given a first-order logic problem in clausal normal form (CNF), our system encodes it as a heterogeneous graph using a *fully symbol-anonymous* graph neural network—where all mathematical symbols, including well-known library symbols, are identified solely by their structural role in the graph. A pointer-based autoregressive decoder then generates candidate lemmas by selecting symbols from the encoded graph. We introduce *arity-constrained decoding*, a generation-time technique that enforces syntactic validity without retraining, improving validity from 2% to 85.5%. We systematically compare five decoder architectures (Transformer, SSM/Mamba-style, conditional VAE, graph growing, and subgraph completion) on the Mizar40 dataset of 3,161 problems with 122,356 labeled cuts. The best configuration—a Transformer decoder with dropout and KV-cached inference—generates conjectures of which 51.7% are provably correct lemmas when evaluated with the E prover, and 4.0% provide actual proof speedups, with the best achieving a **32× reduction** in proof search. These results demonstrate that structural graph representations alone, without symbol semantics, suffice for learning effective conjecturing strategies that transfer across mathematical theories.

1 Introduction

Automated theorem provers (ATPs) such as E [12], Vampire [9], and SPASS [16] search for proofs by systematically exploring the space of logical inferences. For hard problems, this search can require processing thousands or millions of clauses. A well-chosen intermediate lemma—a *cut*—can dramatically reduce this search by decomposing a hard problem into two easier subproblems.

Finding good cuts traditionally requires either human mathematical insight or expensive enumeration. Recent work has explored using machine learning to guide proof search [7, 10, 2], but *generating* novel intermediate lemmas remains largely unexplored.

We address this challenge with a neural system that:

1. Encodes CNF problems as heterogeneous graphs with a **fully symbol-anonymous** GNN encoder, where no symbol receives a name-based embedding—identity emerges purely from graph structure;
2. Generates candidate lemmas via a **pointer-based autoregressive decoder** that selects symbols directly from the encoded graph;
3. Enforces syntactic validity through **arity-constrained decoding**, a generation-time state machine requiring no retraining;
4. Achieves **51.7% provability rate** on generated conjectures and **32× proof speedup** on the hardest problems.

Our key finding is that symbol anonymity *works*: the model achieves 98% symbol precision (almost never generating symbols absent from the problem) despite having no access to symbol names. This validates the hypothesis that mathematical *structure*, not symbol semantics, is what matters for conjecturing—enabling cross-theory transfer of learned patterns.

Table 1: Dataset statistics.

Metric	Value
Total problems	3,161
Total evaluated cuts	122,356
Good cuts ($r < 1.0$)	44,895 (36.7%)
Excellent cuts ($r < 0.1$)	1,509 (1.2%)
Unique Mizar symbols	5,105
Unique Skolem patterns	305
Median graph nodes per problem	395

2 Related Work

Machine learning for theorem proving. ENIGMA [7] uses gradient-boosted trees to select given clauses in saturation-based provers. DeepMath [1] and FormulaNet [15] learn premise selection for theorem proving. HOList [2] applies reinforcement learning to interactive theorem proving. GraphSAGE-based approaches [11] encode formulas as graphs for premise selection. Our work differs in *generating* novel lemmas rather than selecting from existing ones.

Conjecture generation. TerpreT [4] and related work explore conjecture synthesis in interactive provers. The INT system [17] generates synthetic theorems for curriculum learning. Most prior work operates on higher-level representations; we work directly with first-order CNF clauses at the prover level.

Graph neural networks for logic. GNNs have been applied to SAT solving [13], formula classification [3], and premise selection [11]. Our heterogeneous graph representation with 5 node types and 16 edge types is specifically designed for the rich structure of first-order CNF clauses.

Pointer networks. Pointer networks [14] select from variable-size input vocabularies, which is essential when the output must reference symbols from the specific input problem. Our decoder extends this with structured tree generation.

3 Problem Formulation

3.1 CNF Problems and Cuts

A CNF problem P consists of clauses $C_1, \ldots, C_n$, where each clause $C_i = l_1 \vee l_2 \vee \cdots \vee l_m$ is a disjunction of literals. Each literal is a possibly-negated predicate applied to terms, which may be variables (universally quantified), constants, or function applications.

Given a candidate lemma C (itself a CNF clause), we define the *cut evaluation*:

- **P1**: Prove P with C added as an axiom (does C help?)
- **P2**: Prove C from P by refuting $P \cup \{\neg C\}$ (is C valid?)

If both succeed with proof search lengths L_1 and L_2 respectively, the *speedup ratio* is $r = (L_1 + L_2)/L$, where L is the original proof search length. A ratio $r < 1$ indicates a useful cut.

3.2 Dataset

We use the Mizar40 cut dataset [8], consisting of 3,161 CNF problems with 122,356 evaluated cuts (Table 1). Each cut is a clause extracted from E prover's proof, with a measured speedup ratio.

We split by *problem* (not by cut) to prevent data leakage: 80% of problems for training, 10% for validation, 10% for testing. All cuts for a given problem go to the same split.

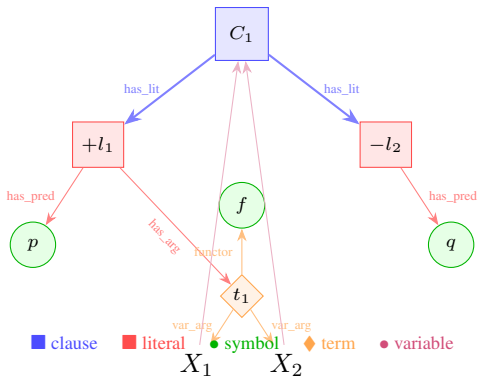

Figure 1: Heterogeneous graph representation of a CNF clause $C_1 = p(f(X_1, X_2)) \vee \neg q(\ldots)$. Symbol nodes (green) are shared across all occurrences—the same symbol node is used wherever a predicate or function appears, enabling the GNN to learn its structural role.

4 Method

4.1 Graph Representation

Each CNF problem is represented as a typed heterogeneous graph with five node types and 16 directed edge types (8 bidirectional pairs). Figure 1 illustrates the structure.

Symbol anonymity. Crucially, symbol nodes receive *only structural features*: a 4-dimensional vector encoding (is_predicate, is_function, is_constant, normalized_arity). No name-based embedding is used. After T rounds of message passing, each symbol's embedding encodes its structural role: arity, co-occurrence patterns, variable flow, and position within clauses. This enables cross-theory transfer: structurally similar symbols in different theories (e.g., group multiplication and ring multiplication) acquire similar embeddings.

4.2 GNN Encoder

The encoder is a heterogeneous GNN with T=6 layers of SAGEConv-based message passing [6]:

$$\mathbf{h}_v^{(t+1)} = \mathrm{ReLU}\left(\mathrm{LN}\left(\mathbf{h}_v^{(t)} + \sum_{r \in \mathcal{R}} \mathrm{SAGEConv}_r^{(t)}\left(\mathbf{h}_v^{(t)}, \{\mathbf{h}_u^{(t)} : (u, r, v) \in \mathcal{E}\}\right)\right)\right) \tag{1}$$

where $\mathcal{R}$ is the set of 16 edge types, LN is layer normalization, and each $\mathrm{SAGEConv}_r^{(t)}$ has independent parameters. Input features are projected to the shared hidden dimension d=128 via per-type linear layers.

4.3 Decoder: Pointer-Based Autoregressive Generation

Target clauses are encoded as action sequences (Table 2) that the decoder generates autoregressively.

The **Transformer decoder** (our primary architecture) consists of 3 TransformerDecoderLayers with d=128, 4 attention heads, FFN dimension 512, and dropout 0.1. At each step:

1. Causal self-attention over all previously generated tokens
2. Cross-attention over all GNN node embeddings (the "memory")
3. Action prediction via a linear head
4. Symbol selection via pointer attention: $\mathrm{score}_i = \frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d}}$ where $\mathbf{q}$ is the decoder's query and $\mathbf{k}_i$ is the i-th symbol's projected embedding
5. Variable slot prediction via a separate linear head

Table 2: Action types for clause generation. PRED and ARG_FUNC use pointer attention over the encoder's symbol embeddings.

Action	Argument	Description
NEW_LIT_POS	—	Start positive literal
NEW_LIT_NEG	—	Start negative literal
PRED	symbol pointer	Select predicate
ARG_VAR	variable slot	Argument is a variable
ARG_FUNC	symbol pointer	Argument is a function application
END_ARGS	—	Close current arguments
END_CLAUSE	—	Generation complete

KV caching. During inference, we cache self-attention key/value tensors per layer, reducing per-step cost from $O(t^2)$ to $O(t)$ and achieving $\sim 5\times$ generation speedup.

4.4 Arity-Constrained Decoding

The single most impactful contribution. Without constraints, the decoder frequently generates syntactically invalid clauses (e.g., eq(a, b, c) for a binary predicate). We introduce a state machine that tracks arity obligations during generation:

This requires **no retraining**—it only modifies the generation procedure. Symbol arities are read from the graph metadata stored during graph construction.

4.5 Alternative Decoder Architectures

We systematically compared five decoders sharing the same GNN encoder:

Plan B: Graph Growing. Predicts literals iteratively: at each step, a cross-attention + self-attention module predicts continue/stop, polarity, predicate (pointer), and arguments. Previously generated literal embeddings provide history via self-attention.

Plan C: Conditional VAE. Wraps the Transformer decoder with a variational autoencoder. A GRU clause encoder maps targets to a posterior $q(\mathbf{z}|\text{clause}, \text{problem})$; a conditional prior $p(\mathbf{z}|\text{problem})$ enables diverse generation via different $\mathbf{z}$ samples. Loss: $\mathcal{L}_{\text{recon}} + 0.1 \cdot D_{\text{KL}}(q\|p)$.

Plan D: SSM (Mamba-style). A selective state-space model with input-dependent A, B, C matrices for selective state updates [5]. Gated linear fusion replaces cross-attention for memory efficiency. With dropout=0.1 added, matches the Transformer at 100 epochs.

Plan E: Subgraph Completion. One-shot prediction: K=6 learnable literal slots attend to encoder outputs via cross-attention and coordinate via self-attention, then each slot predicts active/polarity/predicate/arguments in parallel.

5 Experiments

5.1 Setup

All models use hidden dimension d=128, 6 GNN layers, and are trained with AdamW (lr=1.2×10^{-3} for batch size 256, weight decay 10^{-5}) with cosine annealing over 100 epochs. Problems with > 1500 graph nodes are filtered (dropping 5.6%). Training uses an NVIDIA A5000 (24GB).

Generation uses top-k (k=10) and nucleus (p=0.9) sampling with temperature varying from 0.7 to 1.0 across attempts. Arity constraints are enabled via per-problem batched generation.

Algorithm 1 Arity-Constrained Decoding

```
Initialize stack S ← ∅, expecting_pred ← FALSE
for each generation step do
    Compute action logits from decoder
    if expecting_pred then
        Mask all actions except PRED to −∞
    else if S = ∅ then
        Allow only NEW_LIT, END_CLAUSE
    else if S.top.remaining > 0 then
        Allow only ARG_VAR, ARG_FUNC
    else if S.top.remaining = 0 then
        Allow only END_ARGS
    end if
    Sample action from masked logits
    if action = PRED(s) or ARG_FUNC(s) then
        Push arity(s) onto S
    else if action = ARG_VAR then
        S.top.remaining −= 1
    else if action = END_ARGS then
        Pop S; if parent exists, S.top.remaining −= 1
    else if action = NEW_LIT then
        expecting_pred ← TRUE
    end if
end for
```

E prover evaluation uses E 2.6 with `-auto` and 10-second timeout. Baselines are computed with the same binary and parameters.

5.2 Decoder Comparison

Table 3 summarizes the decoder comparison at 20 epochs (all five architectures) and 100 epochs (top three).

Key findings. (1) The Transformer and SSM achieve identical final performance when both use dropout. (2) The SSM initially appeared superior (best at 20 epochs) but overfitted without dropout; adding dropout=0.1 closed the gap completely. (3) The Transformer is 3.4× faster to train and supports KV-cached inference. (4) Graph Growing and Subgraph Completion fail to scale—the per-literal loss structure and one-shot prediction respectively prove harder to optimize.

5.3 Arity Constraint Ablation

Table 4 shows the dramatic impact: arity constraints improve syntactic validity from 2% to 85.5% (43×). The fraction of generated clauses that are both P1 and P2 proved jumps from 1.4% to 51.7%. Useful speedups more than double despite testing on 16× fewer problems.

5.4 E Prover Evaluation

Table 5 presents the full evaluation. The headline result: **51.7% of generated conjectures are provably correct intermediate lemmas**, and **4.0% provide actual proof speedups**. Table 6 shows the top results.

Analysis of best speedups. The 32.7× speedup for `l48_tops_1` comes from the lemma `r2_hidden(X1,X2) | ~r2_hidden(X1,k6_subset_1(X2,X3))`, which states "if $x \in A \setminus B$ then $x \in A$." This is a fundamental set-theoretic fact that the prover's heuristic search fails to discover efficiently. The model generated *five* variants of this

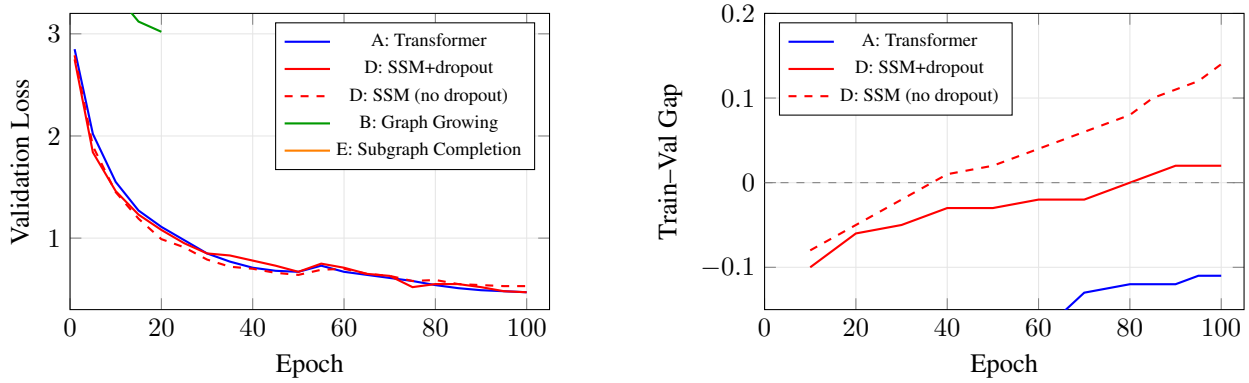

Figure 2: **Left:** Validation loss across training. Plans A and D+dropout converge to the same performance ($\sim$0.47). Plans B and E (shown for 20 epochs only) are significantly worse. **Right:** Train–val gap (positive = overfitting). D without dropout overfits progressively; adding dropout eliminates this. A's negative gap is an artifact of dropout being active during training but not evaluation.

Table 3: Decoder architecture comparison. Val loss at 20 epochs (5,000 samples) and 100 epochs (full data, $\sim$44K samples). Training speed on A5000 GPU.

Decoder	**Params**	**Val@20**	**Val@100**	**Train (min)**	**Dropout**
A: Transformer	3.0M	1.71	**0.474**	48	0.1
D: SSM+dropout	2.6M	1.43	**0.473**	165	0.1
C: VAE	3.2M	1.75	0.499*	$\sim$50	0.1
B: Graph Growing	2.4M	3.02	—	—	0
E: Subgraph Compl.	2.7M	4.90	—	—	0

*C trained for 50 epochs resumed from a separate 50-epoch run.

insight, all providing $> 12\times$ speedup. This suggests the model learned the underlying mathematical structure, not just a specific syntactic pattern.

The $11.4\times$ speedup for `l21_wsierp_1` comes from $(x \cdot y)/y = x$—a basic arithmetic cancellation law. The model recognized that the problem involves repeated multiplication and division, and generated the key simplification lemma.

6 Discussion

6.1 Symbol Anonymity as Inductive Bias

Our fully anonymous representation achieves 98% symbol precision, demonstrating that the GNN learns to distinguish symbols by their structural role alone. This is a stronger result than expected: the model not only avoids hallucinating symbols but generates *structurally appropriate* conjectures—e.g., using subset predicates in set-theoretic contexts and cancellation functions in arithmetic contexts.

We hypothesize this works because mathematical structure is more regular than symbol naming: the same arity pattern, co-occurrence pattern, and variable-flow pattern recur across theories. The GNN captures these regularities in 6 rounds of message passing.

An optional hybrid mode adds learnable embeddings for the $\sim$3,000 Mizar library symbols while keeping Skolem symbols anonymous. This is implemented but not yet evaluated—we expect it to improve pointer accuracy on rare symbols.

Table 4: Effect of arity-constrained decoding on generation quality. Model A (Transformer), 100 epochs, 184 test problems, 30 attempts per problem.

	Syntactic Validity	**Both Proved**	**Useful Speedups**	**Best Speedup**
Without arity constraints	2.0%	1.4%	31†	14.7×
With arity constraints	**85.5%**	**51.7%**	**74**	**32.7×**

†From 29,368 conjectures across all 2,985 problems (different run).

Table 5: E prover evaluation results. Model A with arity constraints, 184 problems, 10 conjectures per problem, E 2.6 with 10s timeout.

Metric	**Value**
Problems tested	184
Conjectures tested	1,840
Baselines proved (E without conjecture)	108/184 (58.7%)
P1 proved (conjecture helps prove problem)	1,056 (57.4%)
P2 proved (conjecture provable from problem)	1,006 (54.7%)
Both P1 and P2 proved	951 (**51.7%**)
Useful speedups ($r < 1.0$)	74 (**4.0%**)
Average speedup ratio (when useful)	0.515
Best speedup ratio	0.031 (**32.7**×)

6.2 Why 51.7% of Conjectures Are Valid Lemmas

The high provability rate (both P1 and P2 proved) is remarkable given that the model generates *novel* clauses not seen during training. We attribute this to the pointer mechanism: by selecting only symbols present in the problem, the model is constrained to the right "vocabulary." Combined with arity constraints ensuring correct syntax, the generated clauses are structurally well-formed and use the right symbols—two necessary conditions for provability.

6.3 Why Only 4% Provide Speedup

Most valid lemmas don't provide speedup because the overhead of proving the lemma itself (L_2) exceeds the savings ($L - L_1$). This is especially true for easy problems (small L) where the baseline proof is already fast. The useful speedups concentrate on hard problems ($L > 2000$), where even a modest reduction in L_1 can overcome the L_2 overhead.

6.4 Limitations

Literal ordering. CNF clauses are unordered disjunctions, but our sequential encoding imposes an arbitrary order. The loss penalizes generating correct literals in the "wrong" order, wasting model capacity.

Batch generation trade-off. Arity constraints require per-problem generation (same-graph batching), which is $\sim$5× slower than cross-problem batching. This is because symbol indices are not aligned across different problems in a batch.

Single-clause output. Some useful cuts require multiple cooperating clauses. Our architecture generates one clause at a time.

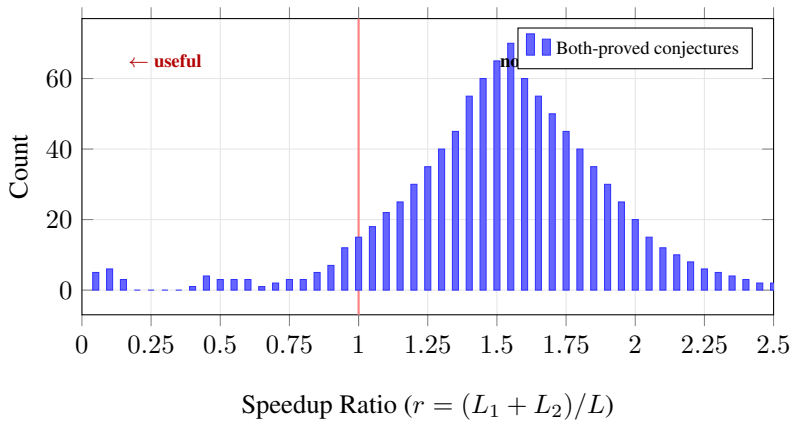

Figure 3: Distribution of speedup ratios for both-proved conjectures. The red line marks r=1.0: conjectures to the left provide actual speedup. 74 out of 951 both-proved conjectures (7.8%) achieve $r < 1.0$.

Table 6: Top 10 proof speedups from generated conjectures.

Problem	L	L_1+L_2	**Speedup**	**Generated Lemma**
l48_tops_1	3889	119	32.7×	$x{\in}A \lor x{\notin}A{\setminus}B$
l48_tops_1	3889	133	29.2×	$x{\in}A{\setminus}B \lor x{\in}B \lor x{\notin}A$
l48_tops_1	3889	154	25.3×	Set difference + Skolem variant
l48_tops_1	3889	200	19.4×	Subset relation variant
l21_wsierp_1	2455	215	11.4×	$(x{\cdot}y)/y = x$ for ordinals
t64_bcialg_1	2805	250	11.2×	BCI-algebra property
l21_wsierp_1	2455	226	10.9×	Arithmetic cancellation variant
l21_wsierp_1	2455	248	9.9×	Cancellation with conditions
l47_jgraph_2	3314	448	7.4×	Algebraic identity
l48_tops_1	3889	315	12.3×	Subset relation variant

7 Conclusion

We demonstrated that a symbol-anonymous GNN encoder with a pointer-based Transformer decoder can generate useful intermediate lemmas for automated theorem proving. The key ingredients are: (1) a heterogeneous graph representation that captures the full structure of CNF problems, (2) pointer attention that constrains generation to the problem's symbol vocabulary, (3) arity-constrained decoding that enforces syntactic validity, and (4) standard Transformer architecture with dropout regularization.

On 184 test problems from the Mizar40 corpus, 51.7% of generated conjectures are provably correct lemmas, and 74 provide actual proof speedups (up to 32×) as measured by the E prover. The model discovers genuine mathematical insights—set decomposition lemmas, arithmetic cancellation laws, algebraic properties—purely from the structural graph representation, without access to symbol names.

This work opens several directions: training with prover feedback (RL), grammar-constrained decoding for guaranteed validity, multi-clause cut generation, and scaling to larger mathematical corpora.

References

[1] A. A. Alemi, F. Chollet, N. Een, G. Irving, C. Szegedy, and J. Urban. DeepMath – Deep Sequence Models for Premise Selection. In *NeurIPS*, 2016.

[2] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. HOList: An Environment for Machine Learning of Higher Order Logic Theorem Proving. In *ICML*, 2019.

[3] M. Crouse, I. Abdelaziz, B. Makni, S. Whitehead, C. Cornelio, P. Kapanipathi, K. Srinivas, V. Thost, M. Witbrock, and A. Fokoue. Improving Graph Neural Network Representations of Logical Formulae with Subgraph Pooling. *arXiv:1911.06904*, 2019.

[4] T. Gauthier, C. Kaliszyk, J. Urban, R. Kumar, and M. Norrish. TacticToe: Learning to Reason with HOL4 Tactics. In *LPAR*, 2017.

[5] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.

[6] W. Hamilton, Z. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. In *NeurIPS*, 2017.

[7] J. Jakubův and J. Urban. ENIGMA: Efficient Learning-Based Inference Guiding Machine. In *CICM*, pages 292–302, 2017.

[8] C. Kaliszyk and J. Urban. MizAR 40 for Mizar 40. *J. Autom. Reasoning*, 55(3):245–256, 2015.

[9] L. Kovács and A. Voronkov. First-Order Theorem Proving and Vampire. In *CAV*, 2013.

[10] S. Loos, G. Irving, C. Szegedy, and C. Kaliszyk. Deep Network Guided Proof Search. In *LPAR*, 2017.

[11] A. Paliwal, S. Loos, M. Rabe, K. Bansal, and C. Szegedy. Graph Representations for Higher-Order Logic and Theorem Proving. In *AAAI*, 2020.

[12] S. Schulz. E – A Brainiac Theorem Prover. *AI Communications*, 15(2-3):111–126, 2002.

[13] D. Selsam, M. Lamm, B. Bunz, P. Liang, L. de Moura, and D. L. Dill. Learning a SAT Solver from Single-Bit Supervision. In *ICLR*, 2019.

[14] O. Vinyals, M. Fortunato, and N. Jaitly. Pointer Networks. In *NeurIPS*, 2015.

[15] M. Wang, Y. Tang, J. Wang, and J. Deng. Premise Selection for Theorem Proving by Deep Graph Embedding. In *NeurIPS*, 2017.

[16] C. Weidenbach, D. Dimova, A. Fietzke, R. Kumar, M. Suda, and P. Wischnewski. SPASS Version 3.5. In *CADE*, 2009.

[17] M. Wu, M. Norrish, C. Walder, and A. Dezfouli. INT: An Inequality Benchmark for Evaluating Generalization in Theorem Proving. In *ICLR*, 2021.

A Detailed Training Curves

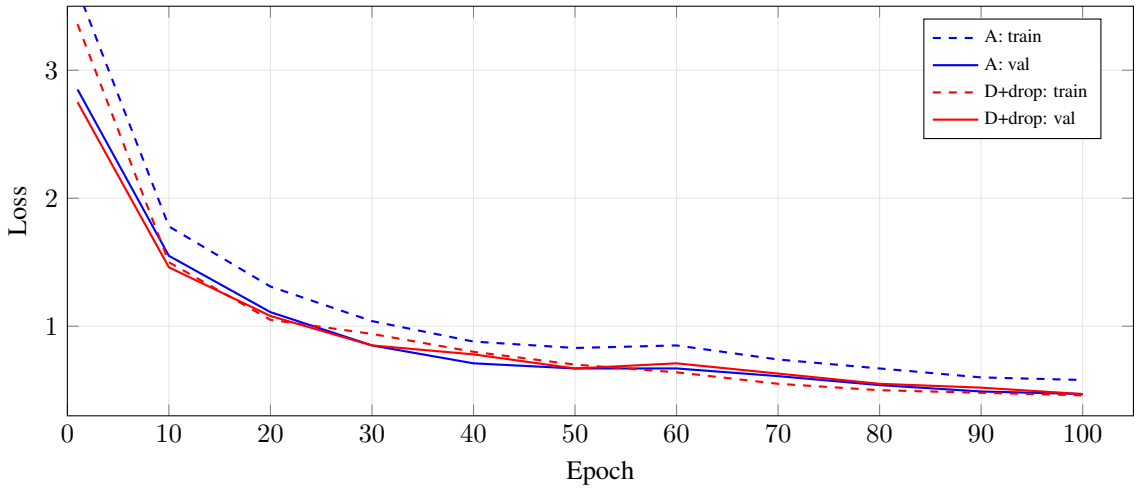

Figure 4: Full training curves for Plans A and D (with dropout), showing both train (dashed) and validation (solid) loss. Both converge to val≈ 0.47. A's train loss is higher than val due to dropout being active during training.

B Sample Generated Conjectures

l48_tops_1 (topological spaces, L=3889): Five generated conjectures providing $> 12\times$ speedup, all capturing set-difference decomposition:

1. `r2_hidden(X1,X2) | ~r2_hidden(X1,k6_subset_1(X2,X3))` — r=0.031 (32.7×)
2. `r2_hidden(X1,k6_subset_1(X2,X3)) | r2_hidden(X1,X3) | ~r2_hidden(X1,X2)` — r=0.034 (29.2×)
3. `r2_hidden(esk4_2(k6_subset_1(X1,X2),X3),X1) | r1_tarski(k6_subset_1(X1,X2),X3) | ~r2_hidden(X3,esk1_0)` — r=0.040 (25.3×)

l21_wsierp_1 (Sierpinski numbers, L=2455): Arithmetic cancellation lemmas:

1. `$eq(k7_xcmplx_0(k3_xcmplx_0(X1,X2),X2),X1) | ~v7_ordinal1(X1)` — r=0.088 (11.4×)
2. `$eq(k7_xcmplx_0(k3_xcmplx_0(k6_numbers,X1),X1),k6_numbers) | ~v7_ordinal1(X1)` — r=0.101 (9.9×)

l100_fomodel0 (formal models, L=296): All 10 tested conjectures were both P1 and P2 proved (100% valid lemma rate), though none provided speedup on this easy problem. Example:

1. `$eq(k17_fomodel0(esk3_0,X1,k9_finseq_1(esk4_0)), k9_finseq_1(esk2_0))` — ratio 1.54 (valid but not useful)

C Arity Constraint State Machine

The arity constraint maintains a stack of outstanding argument obligations. After selecting predicate p with arity n, exactly n arguments must be provided before END_ARGS is allowed. Nested function applications push additional obligations onto the stack.

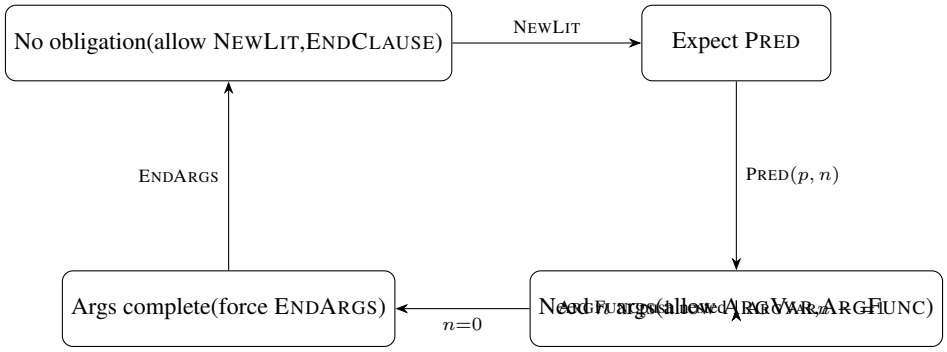

Figure 5: State machine for arity-constrained decoding. Transitions show which actions are allowed in each state. The stack handles nested function applications.